Positional Encoding Alignment for Efficient KV-Cache Reuse in Multi-Document QA

Bogdan Akhmedyanov, Ivan Nikulin
NLP Final Project

Abstract

Large language models are increasingly used in Retrieval-Augmented Generation and Multi-Document Question Answering, where the input often contains several long documents. The main computational bottleneck appears during the prefill stage: the model must process the entire context before generation, while the cost of self-attention grows quadratically with sequence length. This project studies *Positional Encoding Alignment* (PEA), a method that precomputes KV-caches for individual documents and then reuses them inside a combined prompt without fully recomputing the document prefill. The key idea is that for models with Rotary Positional Embeddings (RoPE), the positional phase of cached keys can be corrected by a linear phase shift. We evaluate the method on Qwen2.5-3B-Instruct using MuSiQue Multi-hop QA and a synthetic Needle in a Haystack stress test. Mathematical validation confirms the reversibility of the RoPE shift with an error of $4.77 \cdot 10^{-7}$ and a cosine similarity of 1.0 between the manual shift and native positional shifting. On MuSiQue, the *Aligned* strategy preserves answer quality at the level of *Naive* cache reuse, but better preserves the internal attention structure relative to the full Baseline: 0.9462 versus 0.8378 Spearman attention correlation. In the scaling test, *Aligned* remains correct up to 25 documents, while *Naive* already fails at 20 documents. The average speedup of *Aligned* over full prefill is $11.94\times$.

1 Introduction

Modern LLMs are increasingly used as components of systems that retrieve, combine, and reason over external knowledge. One of the most common settings is Retrieval-Augmented Generation (RAG), where the model receives retrieved documents and answers a question based on them [2]. This reduces reliance on the model's parametric memory, but creates a computational challenge: the more documents are added to the prompt, the more expensive the prefill stage becomes.

In autoregressive inference, the model first processes the full input prompt and builds `past_key_values`, i.e., the KV-cache for all transformer layers. During generation, newly generated tokens can then reuse the already computed keys and values. However, classical KV-cache reuse works most naturally for prefix reuse: if the beginning of the prompt is exactly the same, it can be reused without recomputation. In Multi-Document QA, the same document may be reused across many queries, but it may appear at different positions inside the combined context. This issue is especially important for models with Rotary Positional Embeddings (RoPE) [3]. RoPE encodes token positions by rotating query and key vectors. As a result, cached keys depend on the positions at which the document was originally processed. If a document is cached independently at positions $0, 1, 2, \ldots$, but later inserted after a system prompt and several preceding documents, its true positions in the combined prompt should be shifted. Directly concatenating independently computed KV-caches breaks this positional geometry.

This work studies *Positional Encoding Alignment* (PEA). In the code and result tables, the main strategy is called *Aligned*. The method independently caches documents, but before concatenating their cached keys, it rotates each segment by a phase corresponding to its new offset in the combined prompt. Values are not rotated, because RoPE affects query-key geometry rather than value representations.

The contributions of this project are as follows: (i) we implement a modular pipeline for independent document caching and assembly of a global `DynamicCache`; (ii) we implement a RoPE-

based phase shift for cached keys in Qwen2.5-3B-Instruct [6]; (iii) we validate the mathematical correctness of the shift operation; (iv) we evaluate the method on MuSiQue Multi-hop QA [4] and synthetic Needle in a Haystack; and (v) we analyze the trade-off between quality, latency, and memory as the number of documents increases.

2 Background and Related Work

2.1 Transformer inference and long context

The Transformer is based on self-attention, where each token can interact with other tokens in the input sequence [5]. Standard full self-attention has quadratic complexity in sequence length, which becomes especially costly for long prompts. Methods such as FlashAttention reduce the practical cost of exact attention through IO-aware computation [1], but they do not remove the need to process long contexts during the prefill stage.

The generation pipeline can be divided into two stages. During *prefill*, the model processes the full prompt and builds a KV-cache. During *decode*, the model generates tokens one by one while reusing cached keys and values. If the same document appears repeatedly, its cache can in principle be saved and reused. However, direct reuse is correct only when the cached fragment remains at the same positions or forms an unchanged prefix.

2.2 Rotary Positional Embeddings

RoPE encodes positions by rotating query and key vectors in two-dimensional subspaces [3]. If a pair of coordinates is represented as a complex number, RoPE for position m can be written as:

$$\mathrm{RoPE}_m(z_j) = z_j \cdot e^{im\theta_j}, \tag{1}$$

where z_j is the complex representation of a coordinate pair and θ_j is the frequency of the corresponding dimension.

The key property of RoPE is the linearity of phase shifts:

$$\mathrm{RoPE}_{m+\Delta}(z_j) = z_j \cdot e^{i(m+\Delta)\theta_j} = \mathrm{RoPE}_m(z_j) \cdot e^{i\Delta\theta_j}. \tag{2}$$

Thus, if a key has already been computed for position m, it can be translated to position $m + \Delta$ by multiplying it with an additional phase factor $e^{i\Delta\theta_j}$. This is the mathematical property used by PEA.

2.3 KV-cache reuse

KV-cache reuse is a natural way to accelerate LLM inference: previously processed text fragments can be stored and reused instead of being recomputed. However, direct reuse is non-trivial in RAG and Multi-Document QA, because cached fragments may appear after earlier context and may lose interactions with preceding text. CacheBlend studies cached knowledge fusion and selective recomputation for reusing KV-cache in RAG [7]. In this project, we focus on a narrower but important part of the problem: RoPE-aware positional correction for independently cached document segments.

3 Methodology

3.1 Problem formulation

Let there be a set of documents $D_1, D_2, \ldots, D_n$ and a question Q. In standard full-prefill inference, the model receives one combined prompt:

$$P = S \oplus D_1 \oplus D_2 \oplus \ldots \oplus D_n \oplus Q, \tag{3}$$

where S is the system/user prefix and $\oplus$ denotes token concatenation. In this setting, the token positions of document D_i depend on the lengths of all preceding segments:

$$\mathrm{pos}(D_i) = |S| + \sum_{j=1}^{i-1} |D_j| + [0, |D_i| - 1]. \tag{4}$$

With independent caching, each document is processed separately:

$$\mathrm{pos}_{cached}(D_i) = [0, |D_i| - 1]. \tag{5}$$

Therefore, the cached keys of a document contain RoPE phases corresponding to position 0, although after insertion into the global prompt they should have a non-zero offset.

3.2 Positional Encoding Alignment

PEA corrects this mismatch with an additional RoPE phase shift. If document D_i should start at offset Δ_i in the combined prompt, each cached key is transformed as:

$$K_i^{aligned} = R_{\Delta_i} K_i, \tag{6}$$

where R_{Δ_i} is the RoPE rotation matrix for phase shift Δ_i. In complex form:

$$K_{i,j}^{aligned} = K_{i,j}^{cached} \cdot e^{i\Delta_i\theta_j}. \tag{7}$$

Algorithm 1: PEA for document cache reuse
Input: documents $D_1, \ldots, D_n$, prefix S, question suffix Q, model M with RoPE.
1. Precompute KV-cache for S.
2. Precompute KV-cache for each D_i independently.
3. Initialize global cache with cache(S).
4. Set current offset $= |S|$.
5. For each document D_i:
   a. take cached K_i, V_i;
   b. rotate K_i by RoPE phase shift for current offset;
   c. keep V_i unchanged;
   d. concatenate shifted K_i and original V_i;
   e. update offset $+= |D_i|$.
6. Run generation on Q using assembled cache.
Output: generated answer.

Figure 1: Pseudocode of Positional Encoding Alignment.

Values are left unchanged:

$$V_i^{aligned} = V_i^{cached}. \tag{8}$$

This distinction is important: RoPE affects query-key dot products, while values serve as content representations aggregated by attention weights. Therefore, positional alignment is applied only to K.

In the implementation, the `shift_cache` function takes a list of KV tensors across layers, computes `inv_freq`, builds `cos` and `sin` for the given offset, casts K to `float32` for numerical stability, applies the rotation, and casts the result back to the original dtype. V is returned without modification.

3.3 Strategies

We compare three inference strategies. *Baseline* is the standard full-prefill run over the complete prompt. It is used as the reference for deep metrics. *Aligned* is the proposed PEA strategy: independent document prefill, RoPE shifting of cached keys, and KV concatenation. *Naive* is an ablation control: it uses independent document prefill and simple KV concatenation without positional correction. Therefore, the difference between *Aligned* and *Naive* isolates the contribution of positional alignment.

3.4 Implementation details

Experiments are conducted with the Qwen2.5-3B-Instruct model. We use Hugging Face Transformers and its DynamicCache interface. The run configuration uses eager attention, disables 4-bit quantization, and stores tensors in bf16. The hardware environment is a single NVIDIA V100 32GB GPU. Eager attention is used because it allows us to extract attention weights for deep metrics. Generation uses greedy decoding with a maximum of 20 new tokens.

4 Experimental Setup

4.1 Datasets

The first dataset is MuSiQue, a multi-hop QA benchmark constructed through the composition of single-hop questions [4]. We use the `answerable` subset and the `validation` split. We select 800 examples with more than one supporting paragraph. This matches the Multi-Document QA setting, where the model should use multiple relevant evidence fragments rather than a single passage.

The second dataset is a synthetic Needle in a Haystack stress test. For each value of `num_docs`, we create a set of filler documents and insert a secret code into the middle document. The model is asked: `What is the secret code?` This benchmark does not test complex reasoning, but it is useful for stress-testing long context behavior and checking whether cache reuse breaks as the number of documents grows.

4.2 Metrics

For answer quality on MuSiQue, we use Exact Match and token-level F1. To compare the internal behavior of the model, we compute deep metrics relative to the Baseline: *Top1 Agreement*, KL divergence over the top-k tokens of the Baseline distribution, the rank of the first gold answer token, and Spearman correlation between Baseline attention weights and the tested strategy. For the scaling test, we measure latency, speedup, memory usage, context length, and EM.

4.3 Mathematical validation

Before the main experiments, we validate the RoPE shift operation. The validation consists of three checks. First, reversibility: a random `float32` key tensor is shifted by $+\Delta$ and then by $-\Delta$. The maximum absolute reconstruction error is $4.77 \cdot 10^{-7}$. Second, attention invariance: for real text, we compare the attention score before shifting with the score after a consistent shift of both query and key. The difference is $6.10 \cdot 10^{-5}$. Third, native consistency: the manual shift is compared with the key tensor that the model would produce when given explicit shifted `position_ids`. The maximum absolute differ-

ence is 0.03085, and the cosine similarity is 1.0. The check is performed on a synthetic tensor and on a representative real KV layer. Since the same deterministic RoPE operation is applied independently to cached keys in every layer, this validates the implemented phase shift.

5 Results

5.1 Accuracy on MuSiQue

Strategy	EM	F1	Top1	AttnCorr	KL	GTRank
Aligned	37.38	49.80	71.75	0.9462	1.4640	2.0
Naive	37.12	49.77	72.62	0.8378	1.4144	2.0

Table 1: MuSiQue results on 800 validation examples. Baseline is used only as the reference run for deep metrics. EM, F1 and Top1 are reported in percent.

At the level of external QA metrics, *Aligned* and *Naive* look almost identical. EM differs by only 0.26 percentage points, and F1 is nearly the same. This is an important result: short answer-level metrics on MuSiQue do not reveal a dramatic difference between positionally corrected cache reuse and naive cache reuse.
However, the deep metrics reveal a more informative picture. The most important difference is attention correlation. *Aligned* reaches 0.9462, while *Naive* reaches only 0.8378. This means that PEA better preserves the internal attention pattern of the model relative to the full Baseline. In other words, even when the final answer sometimes matches, *Naive* distorts the model's mechanism of attending to context more strongly.

5.2 Scaling stress-test

Docs	Baseline	Aligned	Naive
5	1.0	1.0	1.0
10	1.0	1.0	1.0
15	1.0	1.0	1.0
20	1.0	1.0	0.0
25	1.0	1.0	0.0
30	1.0	0.0	0.0
35	1.0	0.0	0.0

Table 2: Exact Match in synthetic Needle in a Haystack by number of documents.

The scaling stress-test shows a clearer difference between strategies. Baseline maintains EM = 1.0 in all conditions, as expected: the model receives the full prompt and recomputes the full context in the standard way. *Naive* works on shorter contexts up to 15 documents, but completely fails at 20 documents. This confirms that simple KV-cache concatenation without positional correction is not a robust solution.
Aligned maintains EM = 1.0 up to 25 documents. It also fails at 30 and 35 documents, which is important to state explicitly: the method is not a full replacement for full prefill on all long contexts. However, it substantially shifts the failure point: *Naive* fails at 20 documents, while *Aligned* remains reliable for a longer context. The latency trend is shown in Figure 2.

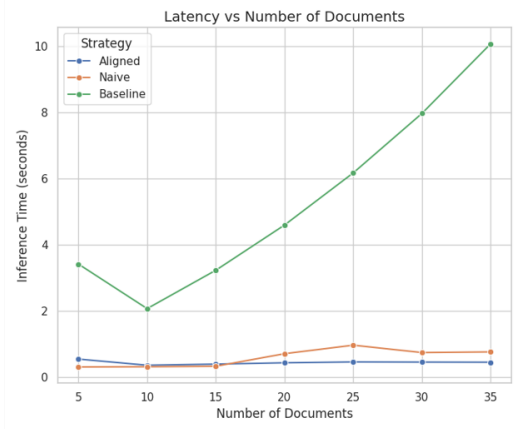

Figure 2: Latency versus the number of documents. Baseline grows with context size, while cache reuse strategies remain much faster.

5.3 Computational efficiency

Strategy	Time, sec	Speedup	EM
Baseline	5.364	1.00×	1.000
Aligned	0.446	11.94×	0.714
Naive	0.593	9.13×	0.429

Table 3: Average performance on synthetic Needle in a Haystack.

The average latency of Baseline is 5.364 seconds, while the average latency of *Aligned* is 0.446 seconds. This gives an average speedup of 11.94×. *Naive* also accelerates inference relative to Baseline, but its average accuracy is lower: 0.429 compared with 0.714 for *Aligned*. Figure 3 summarizes this quality-speed trade-off.

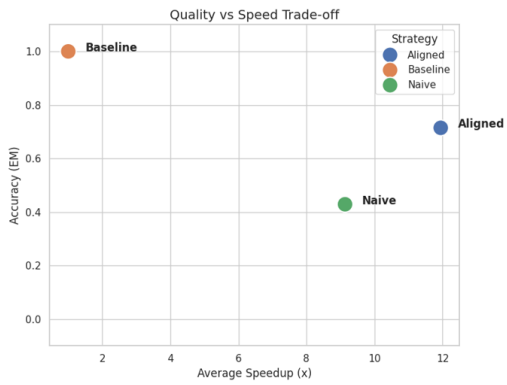

Figure 3: Quality-speed trade-off. *Aligned* is much faster than Baseline while preserving higher average EM than *Naive*.

It is important to clarify the computational claim. The speedup does not mean that all LLM computation becomes linear. Rather, it comes from reusing the document prefill. If documents have already been cached or are reused across many queries, a new request does not need to run the entire document context through the model again. Instead, the system applies a linear operation over the KV-cache to perform RoPE alignment, concatenates KV tensors, and then runs the model on the question suffix and generated tokens.
KV-cache memory grows approximately linearly with context length: from 49.1 MB at 1393 tokens to 325.4 MB at 9253 tokens. This is expected because each layer stores key and value representations for every token. Therefore, PEA shifts the bottleneck from repeated computation to storage and memory bandwidth, which is a typical trade-off in cache reuse systems. Figure 4 shows the observed memory trend.

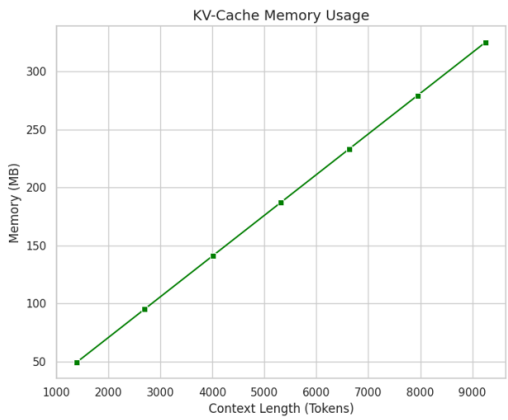

Figure 4: KV-cache memory usage versus context length. The observed growth is close to linear.

6 Discussion

6.1 Why Naive works on short contexts

Naive preserves quality on a small number of documents. A possible explanation is that at short distances the model can partially compensate for positional errors through content representations, local attention patterns, and the robustness of instruction-tuned models to small prompt perturbations. In addition, the Needle task is relatively simple: the model only needs to extract the explicit string `The secret code is ...`, rather than perform complex multi-hop reasoning. However, as the number of documents grows, the positional error accumulates. All independently cached documents look as if they start at position 0. As a result, relative distances between the question and documents become incorrect. For RoPE, this is critical because the attention score depends on the phase relationship between query and key. This explains why *Naive* sharply degrades at 20 documents.

6.2 Why Aligned is more stable

Aligned corrects the part of the error that is caused by positional encoding. Cached keys of each document are moved to the positions they would have had in the combined prompt. This allows question query tokens to interact with document keys in a more correct RoPE geometry. This explains the higher attention correlation on MuSiQue and the later degradation in the scaling test.
At the same time, *Aligned* does not modify values. This is not an implementation limitation, but a property of RoPE: positional information is injected into queries and keys, while values serve as content vectors aggregated by attention weights.

6.3 Main limitation: missing cross-document attention

The main limitation of the method is not the RoPE shift itself, but independent prefill. When documents are processed separately, tokens inside D_i cannot attend to tokens from $D_1, \ldots, D_{i-1}$. In the full Baseline, this interaction is available because the whole prompt is processed as one sequence. PEA restores the positional geometry of keys, but it does not recompute document hidden states with access to preceding documents.

Therefore, PEA is best suited for scenarios where documents can be treated as relatively independent evidence chunks and the main cross-document reasoning happens during question processing and generation. If the task requires strong integration between documents already during prefill, full Baseline or partial recomputation may be more reliable.

This is where methods such as CacheBlend are relevant [7]. They recognize that direct reuse of cached fragments can be incorrect due to missing cross-attention with preceding text, and they partially recover this interaction through selective recomputation. In this sense, PEA can be viewed as a cheap and mathematically clean positional correction layer that could potentially be combined with more expensive partial recomputation methods.

6.4 Practical implications

In practice, the method is most useful for RAG systems where the same document collection is reused across many queries. Documents can be precomputed offline, their KV-caches can be stored, and the context for a specific question can be assembled quickly. In this regime, the cost of document prefill is amortized across many requests.

For a one-off query with entirely new documents, the benefit is smaller because the documents still need to be cached first. Therefore, PEA should be viewed as a method for accelerating repeated document reuse, not as a universal acceleration technique for every single-query long-context inference scenario.

7 Conclusion

This project studied *Positional Encoding Alignment* for efficient KV-cache reuse in Multi-Document QA. The method is based on a property of RoPE: a positional shift can be implemented as a linear phase rotation of cached keys. This makes it possible to cache documents independently at position 0 and later move them to the correct positions inside a combined prompt. Mathematical validation confirms the correctness of the operation: the RoPE shift is reversible with an error on the order of 10^{-7}, attention scores are preserved with a small error, and the manual shift matches native positional shifting with a cosine similarity of 1.0. On MuSiQue, *Aligned* achieved almost the same EM/F1 as *Naive*, but preserved the attention structure relative to Baseline much better. On synthetic Needle in a Haystack, *Aligned* was substantially more stable than *Naive*: *Naive* failed at 20 documents, while *Aligned* remained correct up to 25 documents. The average speedup of *Aligned* over full Baseline was $11.94\times$.

The main conclusion is that RoPE-aware positional alignment is a cheap and effective operation for cache reuse. It does not fully replace full prefill because it does not restore cross-document attention under independent caching, but it substantially improves the stability of KV-cache reuse compared with naive concatenation. Future work may extend this method to system prompt caching, hierarchical knowledge bases, hybrid RAG systems, and combinations with selective recomputation methods such as CacheBlend.

References

[1] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359.

[2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

[3] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced transformer with rotary position embedding. *Preprint*, arXiv:2104.09864.

[4] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

[6] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 23

others. 2024. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

[7] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2024. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *Preprint*, arXiv:2405.16444.